



SWEN 221

A surreal, dreamlike landscape. In the foreground, a person stands on a dark, rocky outcrop, looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, and blue. A bright, glowing horizon line suggests a sunrise or sunset. In the upper right, a comet streaks across the sky, leaving a trail of light. The overall atmosphere is ethereal and otherworldly.

Marco Servetto
CO258

www.youtube.com/MarcoServetto

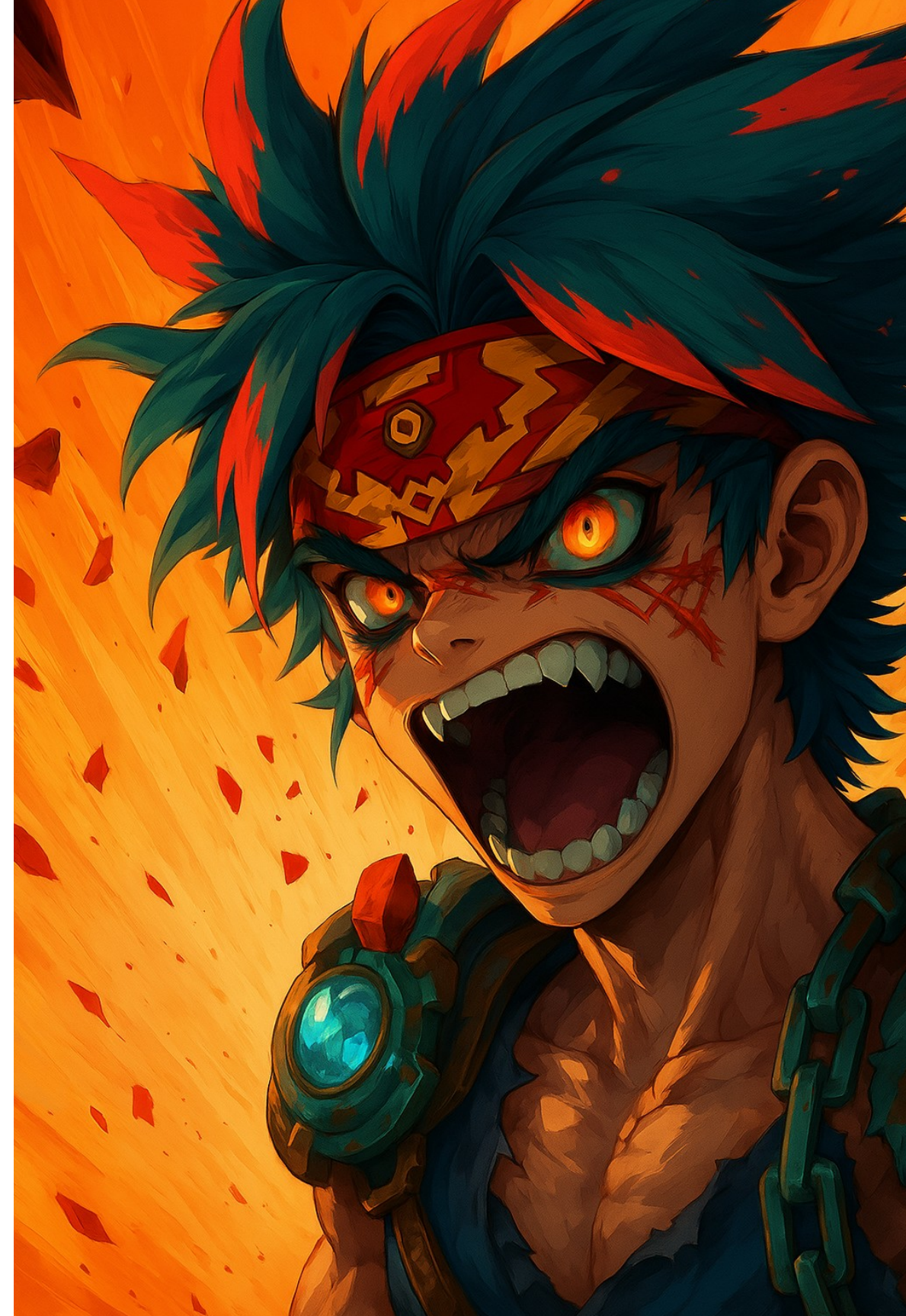
Optional

- Do you know that feeling, where you are happily coding, all is working fine... and then...
- Null Pointer exception!
- You just called a method, supposed to return a Person, but... null was returned.
- It feels as if the method signature lied to you...



Optional

- Sadly, that is the reality of Java.
- Null is in the language, and can always be returned or passed.
- From Java 8 we have
 - `Optional<T>`
- It does not solve null pointers, but if used consistently can make it a little less pervasive.



Optional, an alternative to null

- use `Optional<T>` and be more explicit!
 - `isPresent` + `get` if you are an old style programmer

```
public static void foo(Optional<Person> p){  
    //Persons have name and age,  
    if(p.isPresent()){//but I'm not sure if I have a person  
        System.out.println(p.get().name);//sure now!  
    }  
}
```

- `ifPresent` if you are a professional programmer!

```
public static void foo(Optional<Person> p){  
    p.ifPresent( sureP-> System.out.println(sureP.name) );  
}
```



Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));  
Optional<Person> nope= Optional.empty(); //or Optional.ofNullable(null);
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));  
Optional<Person> nope= Optional.empty(); //or Optional.ofNullable(null);  
//this does not create a new object
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));  
Optional<Person> nope= Optional.empty(); //or Optional.ofNullable(null);  
//this does not create a new object  
  
if (marco.isPresent()){... marco.get();... };
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));  
Optional<Person> nope= Optional.empty(); //or Optional.ofNullable(null);  
//this does not create a new object
```

```
if (marco.isPresent()){... marco.get();... };
```

```
marco.ifPresent(m->...);
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

Optional: create and manipulate

```
Optional<Person> marco= Optional.of(new Person("Marco",34));  
Optional<Person> nope= Optional.empty(); //or Optional.ofNullable(null);  
//this does not create a new object
```

```
if (marco.isPresent()){... marco.get();... };
```

```
marco.ifPresent(m->...);
```

```
nope.get();//Dynamic error
```

- Optional is a proxy.
- Shows the programmer intention in the types.
- If used consistently instead of nulls
-> code more readable and predictable.
- No agreement on when
it is good to use null and when it is good to use Optional

More Optional lambda methods:

```
//Inside of Optional<T>{...}
```

```
<U> Optional<U> map(Function<? super T,? extends U> mapper)
```

If a value is present, returns an Optional describing (as if by ofNullable(T)) the result of applying the given mapping function to the value, otherwise returns an empty Optional.

```
<U> Optional<U> flatMap(Function<? super T,? extends Optional<? extends U>> mapper)
```

If a value is present, returns the result of applying the given Optional-bearing mapping function to the value, otherwise returns an empty Optional.

```
Optional<T> filter(Predicate<? super T> predicate)
```

If a value is present, and the value matches the given predicate, returns an Optional describing the value, otherwise returns an empty Optional.

```
void ifPresentOrElse(Consumer<? super T> action, Runnable emptyAction)
```

If a value is present, performs the given action with the value, otherwise performs the given empty-based action.

Simplified signatures

```
//Inside of Optional<T>{...}
```

```
<U> Optional<U> map(Function<T,U> mapper)
```

If a value is present, returns an Optional describing (as if by ofNullable(T)) the result of applying the given mapping function to the value, otherwise returns an empty Optional.

```
<U> Optional<U> flatMap(Function<T,Optional<U>> mapper)
```

If a value is present, returns the result of applying the given Optional-bearing mapping function to the value, otherwise returns an empty Optional.

```
Optional<T> filter(Predicate<T> predicate)
```

If a value is present, and the value matches the given predicate, returns an Optional describing the value, otherwise returns an empty Optional.

```
void ifPresentOrElse(Consumer<T> action, Runnable emptyAction)
```

If a value is present, performs the given action with the value, otherwise performs the given empty-based action.

More Optional lambda methods:

```
//Inside of Optional<T>{...}
```

```
Optional<T> or(Supplier<? extends Optional<? extends T>> supplier)
```

If a value is present, returns an Optional describing the value,
otherwise returns an Optional produced by the supplying function.

```
T orElse(T other)
```

If a value is present, returns the value,
otherwise returns other.

```
T orElseGet(Supplier<? extends T> supplier)
```

If a value is present, returns the value,
otherwise returns the result produced by the supplying function.

```
<X extends Throwable> T orElseThrow(Supplier<? extends X> exceptionSupplier)
```

If a value is present, returns the value, otherwise throws an exception produced by the exception
supplying function.

- All the methods of optional discussed in class can be needed in the term tests!

Simplified signatures

//Inside of Optional<T>{...}

Optional<T> or(Supplier<Optional<T>> supplier)

If a value is present, returns an Optional describing the value,
otherwise returns an Optional produced by the supplying function.

T orElse(T other)

If a value is present, returns the value,
otherwise returns other.

T orElseGet(Supplier<T> supplier)

If a value is present, returns the value,
otherwise returns the result produced by the supplying function.

<X extends Throwable> T orElseThrow(Supplier<X> exceptionSupplier)

If a value is present, returns the value, otherwise throws an exception produced by the exception
supplying function.

- All the methods of optional discussed in class can be needed in the term tests!


```
record PoliceId(){}  
record Person(String name, int age){}  
...  
Optional<Person> selectPerson(int id){ ... }  
  
Optional<String> canDrive(int id){ ... }
```

```
record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    ....
    ....
    ....
}
...
```

```
record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        ....
        ....
}
...
```

```
record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        ....
}
...
```

```
record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
        //Will return an Optional.empty() if person.name == null
}
...
```



```

record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
        //Will return an Optional.empty() if person.name == null
}
...

Optional<PoliceId> selectPoliceman(int id, String name){ ... }

boolean canDriveFast(int id){
    return ....
        ....
        ....
        ....
}

```

```

record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
    //Will return an Optional.empty() if person.name == null
}
...

Optional<PoliceId> selectPoliceman(int id, String name){ ... }

boolean canDriveFast(int id){
    return canDrive(id) //Optional<String>
        ....
        ....
        ....
        ....
}

```

```

record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
        //Will return an Optional.empty() if person.name == null
}
...

Optional<PoliceId> selectPoliceman(int id, String name){ ... }

boolean canDriveFast(int id){
    return canDrive(id) //Optional<String>
        .flatMap(name->selectPoliceman(id,name)) //Optional<PoliceId>
        ....
        ....
        ....
}

```

```

record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
        //Will return an Optional.empty() if person.name == null
}
...

Optional<PoliceId> selectPoliceman(int id, String name){ ... }

boolean canDriveFast(int id){
    return canDrive(id) //Optional<String>
        .flatMap(name->selectPoliceman(id,name)) //Optional<PoliceId>
        .map(policeId->true) //Optional<Boolean>
        //either Optional.of(true) or Optional.empty()
    ....
}

```

```
record PoliceId(){}
record Person(String name, int age){}
...
Optional<Person> selectPerson(int id){ ... }

Optional<String> canDrive(int id){
    return selectPerson(id) //Optional<Person>
        .filter(person->person.age() > 20) //Optional<Person>
        .map(person->person.name()); //Optional<String>
        //Will return an Optional.empty() if person.name == null
}
...

Optional<PoliceId> selectPoliceman(int id, String name){ ... }

boolean canDriveFast(int id){
    return canDrive(id) //Optional<String>
        .flatMap(name->selectPoliceman(id,name)) //Optional<PoliceId>
        .map(policeId->true) //Optional<Boolean>
        //either Optional.of(true) or Optional.empty()
        .orElse(false); //boolean
}
```

Example for optional

//Use Optional.orXXX methods to decompose long methods

```
Person largeMethod(){  
  
    //doStuff1  
    /*..lots of code here containing some if-returns */  
    if (cond1){ return /*..*/; }  
  
    //doStuff2  
    /*..lots of code here containing some if-returns */  
    if (cond2){ return /*..*/; }  
  
    //doStuff3  
    /*..lots of code here containing some if-returns */  
    if (cond3){ return /*..*/; }  
  
    //fail in the end  
    throw new MyGoodError();  
}
```


Decompose methods with Optional.orXX

```
Person largeMethod(){
    //doStuff1
    /*..lots of code here containing some if-returns */if(cond1){ return /*..*/; }
    //doStuff2
    /*..lots of code here containing some if-returns */if(cond2){ return /*..*/; }
    //doStuff3
    /*..lots of code here containing some if-returns */if(cond3){ return /*..*/; }
    //fail in the end
    throw new MyGoodError();
}
```

---BECOMES---

```
Person largeMethod(){
    return doStuff1()
        .or(()->doStuff2())
        .or(()->doStuff3())
        .orElseThrow(MyGoodError::new);
}
//Now we can define doStuff1-2-3 methods returning Optionals<Person>
```

Decompose methods with Optional.orXX

```
Person largeMethod(){
    return doStuff1()
        .or(()->doStuff2())
        .or(()->doStuff3())
        .orElseThrow(MyGoodError::new);
}
Optional<Person> doStuff1(){..}
Optional<Person> doStuff2(){..}
Optional<Person> doStuff3(){..}
```

```
Person largeMethodNoError() {//Variant to return a result instead of an error.
    return doStuff1()
        .or(()->doStuff2())
        .or(()->doStuff3())
        .orElseGet(()->doStuffSure());
}
Person doStuffSure(){..}
```

Optional vs exceptions

```
//Contract: precondition: list must be not empty, 'null' not in ps
public static Person oldest1(List<Person> ps){
    assert !ps.isEmpty();
    Person res= ps.getFirst();//new in Java 22 :-)
    for (var pi : ps){ res = pi.age() > res.age() ? pi : res; }
    return res;
}
```

```
//Contract: precondition: 'null' not in ps
public static Optional<Person> oldest2(List<Person> ps){
    if (ps.isEmpty()) { return Optional.empty(); }
    return Optional.of(oldest1(ps));
}
```

- Two good methods. But they are different methods with different contracts.
- Both versions have their value and their use.

Optional vs exceptions

//Bad. Why this is likely bad?

```
Optional<Path> writeFile(String fileName, String content){  
    var p= Path.of(fileName);  
    try { return Optional.of(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return Optional.empty(); }  
}
```

//User code

Optional vs exceptions

//Bad. Why this is likely bad?

```
Optional<Path> writeFile(String fileName, String content){  
    var p= Path.of(fileName);  
    try { return Optional.of(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return Optional.empty(); }  
}
```

//User code

//bla bla bla

```
String myAssignment= "..";
```

```
//bla bla bla myAssignment+=... bla bla bla
```

```
writeFile("backup.txt",myAssignment);
```

```
/bla bla ignoring the result
```

Optional vs exceptions

//Bad. Why this is likely bad?

```
Optional<Path> writeFile(String fileName, String content){  
    var p= Path.of(fileName);  
    try { return Optional.of(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return Optional.empty(); }  
}
```

//User code

//bla bla bla

String myAssignment= "..";

//bla bla bla myAssignment+=... bla bla bla

writeFile("backup.txt",myAssignment);

/bla bla ignoring the result

- Here we will not discover that we lost the assignment!
- Having an honest return type does not matter if we can ignore it.

How to delay exceptions without Optional

//good variant

```
<R> R writeFile(String fileName, String content,  
    Function<Path,R> success, Function<IOException,R> failure){  
    var p= Path.of(fileName);  
    try { return success.apply(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return failure.apply(e); }  
}
```

//User code

How to delay exceptions without Optional

//good variant

```
<R> R writeFile(String fileName, String content,  
    Function<Path,R> success, Function<IOException,R> failure){  
    var p= Path.of(fileName);  
    try { return success.apply(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return failure.apply(e); }  
}
```

//User code

//bla bla bla

String myAssignment= "..";

//bla bla bla myAssignment+=... bla bla bla

writeFile("backup.txt",myAssignment,

p-> p,

e->{ throw new UncheckedIOException(e); });

//now the user *have* to chose what happens.

//here we get an error

How to delay exceptions without Optional

//good variant

```
<R> R writeFile(String fileName, String content,  
    Function<Path,R> success, Function<IOException,R> failure){  
    var p= Path.of(fileName);  
    try { return success.apply(Files.write(p, content.getBytes())); }  
    catch (IOException e) { return failure.apply(e); }  
}
```

//User code

//bla bla bla

String myAssignment= "..";

//bla bla bla myAssignment+=... bla bla bla

writeFile("backup.txt",myAssignment,

p-> p,

e->{ throw new UncheckedIOException(e); });

//now the user *have* to chose what happens.

//here we get an error

boolean success= writeFile("backup.txt", myAssignment, p->true, e->>false);

//here we get a boolean



Next: Finally, streams!